

Skapa strängar

Man kan skapa instanser av klassen `string` på flera olika sätt. De vanligaste metoderna är att antingen skapa en tom instans eller så skapa en instans och initiera den direkt med en sträng:

```
string variabelnamn1;  
string variabelnamn2 = "värde";
```

Den första deklarationen skapar en tom sträng medan den andra skapar en sträng med ett fördefinierat värde. En kopia av det fördefinierade värdet görs. Strängar kan även skapas som kopior av varandra, enligt följande:

```
string S1 = "Bilbo";  
string S2 = S1;
```

Strängen `S2` är nu en *kopia* av `S1`, och all manipulering av `S2` påverkar inte `S1`. Ifall man vill skapa en ny sträng och fylla denna med ett antal av ett visst specifikt tecken kan detta även göras. För att skapa en sträng innehållande 10 `A` kan följande användas:

```
string S ( 10, 'A' );
```

Ibland kan det vara användbart att kunna skapa en sträng av en del av en annan sträng. Man använder då ordet *substräng*. En substräng kan skapas genom att till konstruktorn för `string` ge som parametrar den ursprungliga strängen som man vill ha en substräng ur, indexet där man vill börja substrängen samt antalet tecken. Ett exempel på en substrängsinitiering:

```
string S1 = "Hello world";  
string S2 ( S1, 6, 5 ); // starta från 'w' och kopiera fem tecken
```

Även då man initierar från en substräng görs en kopia av tecknen. I exemplet ovan har `S2` ingenting längre gemensamt med `S1` efter initierringen, annat än att den råkar innehålla delvis samma text som i `S1`.

Längden av strängar

För att få reda på längden av en sträng kan man använda sig av metoderna `size()` eller `length()`. Båda ger samma värde, d.v.s. antalet tecken i strängen. T.ex.:

```
#include <iostream>  
#include <string>  
  
int main () {  
    string S1 = "Hello world";  
    string S2;  
  
    // skriv ut strängens längd  
    cout << "size()    : " << S1.size () << endl;  
    cout << "length() : " << S1.length () << endl;  
}
```

Man kan alltså använda antingen `length()` eller `size()`. En tom sträng har alltid längden 0.

[Förutgående](#)
Stränghantering

[Hem](#)
[Upp](#)

[Nästa](#)
Accessera enskilda tecken